Optical Engineering

OpticalEngineering.SPIEDigitalLibrary.org

Dual pseudorandom array technique for error correction and hole filling of color structured-light three-dimensional scanning

Udaya Wijenayake
Soon-Yong Park

SPIE.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

Optical Engineering 54(4), 043109 (April 2015)

Dual pseudorandom array technique for error correction and hole filling of color structured-light three-dimensional scanning

Udaya Wijenayake and Soon-Yong Park*
Kyungpook National University, School of Computer Science and Engineering, 80 Daehak-ro, Buk-gu, Daegu 702-701, Republic of Korea

Abstract. The accurate and dense real-time acquisition of three-dimensional (3-D) data using a low-cost structured light system remains an ongoing topic in the computer vision community, as it is difficult to achieve all these features simultaneously. Among several techniques, the pseudorandom array technique is widely used in real-time 3-D scene capturing, as it tends to concentrate the entire coding scheme into a single pattern. However, existing pseudorandom array decoding methods cannot decode a given symbol in real time when missing neighbors exist. As a solution, we propose a dual pseudorandom array encoding and decoding method and a hole-filling method, which can improve the reconstruction accuracy and time. We experimentally compared our method with several others to verify that our method captures 3-D scenes quickly and densely. *© 2015 Society of Photo-Optical Instrumentation Engineers (SPIE)* [DOI: 10.1117/1.OE.54.4.043109]

Keywords: structured light; spatial neighborhood; pseudorandom arrays; dual encoding; dual decoding; hole filling.

Paper 141997 received Dec. 30, 2014; accepted for publication Mar. 30, 2015; published online Apr. 27, 2015.

1 Introduction

The techniques used in three-dimensional (3-D) surface reconstruction and measurements received increasing attention in the computer vision community during the past few decades because of emerging technologies such as range sensors, autonomous vehicles, biometrics, industrial inspection systems, 3-D object modeling, and device-free gaming interfaces. The 3-D measurement techniques introduced in the past years can be divided into two main categories: contact and noncontact.

Contact techniques use a probe connected to a mechanical system that can move three-dimensionally. A main disadvantage of this method is that the object can be damaged or modified during the contact scanning. The damage is very significant when delicate or valuable objects, such as historical artifacts, are scanned. Slow performance and high cost are additional disadvantages as compared with other scanning methods.

Three-dimensional scanning methods, such as laser scanning, time of flight, stereo vision, silhouette carving, shape from motion, and structured light, are categorized as noncontact methods. These noncontact methods can be further divided into two different categories: active and passive. Passive methods do not interfere at all with the scene that is to be reconstructed. Only the sensors, typically image sensors, are used to measure the radiance reflected or emitted from the object's surface. Stereo, motion, shading, and silhouettes are some of the passive methods that take images captured from two or more points of view as the input. The main problem that these methods present is that the reconstruction is sparse because of the difficulty in finding satisfactory correspondences between multiple views. In contrast, active methods actively interfere with the object that is to be reconstructed by emitting radiance toward it and measuring its reflections.

The structured light method, in which a light pattern is emitted onto a scene to create artificial correspondences by giving specific codewords to every unitary position on the image, is such an active scanning method.[1] Single or multiple cameras can be used to capture the illuminated scene, and the 3-D information of the scene surface can be found by solving the artificial correspondences. Structured light techniques can be classified into three main categories according to the coding strategy and the pattern structure: time multiplexing, direct codification, and spatial codification.[2] In time multiplexing methods, a sequence of simple patterns is projected onto the surface and the codeword of a given pixel is formed by the illuminance values. The main drawback of these methods is the weakness in capturing dynamic scenes, as the projection of multiple patterns is very time consuming. In direct codification methods, the entire codeword of a given point is contained in a unique pixel and is represented by its color value. High pixel resolution projectors and a large range of color values are used to obtain high resolutions; however, their applicability is limited because of their high sensitivity to noise and light variations and low sensitivity to surface changes. Spatial neighborhood techniques tend to integrate the entire coding scheme in a single pattern so that it can be used in the dynamic scene capturing. However, solving the correspondences in spatial neighborhood techniques is not stable due to several reasons such as pattern distortion caused by the color and shape of the object surface. In this paper, a new spatial neighborhood technique that solves the aforementioned problem is proposed using dual pseudorandom arrays.

The paper is organized as follows. First, an introduction to spatial neighborhood techniques and a review of related

*Address all correspondence to: Soon-Yong Park, E-mail: sypark@knu.ac.kr

0091-3286 /2015/$25.00 © 2015 SPIE

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

works are presented in Sec. 2. A brief overview of the proposed method, along with some advantages compared to existing techniques, is presented in Sec. 3. The proposed dual pseudorandom array encoding method is comprehensively described in Sec. 4. The proposed dual pseudorandom array decoding method along with the hole-filling algorithm implemented to correct erroneous reconstruction results is described in Sec. 5. The experimental results and further discussions are presented in Sec. 6. Accuracy analysis, comparison with existing methods, and some of the limitations of the proposed method are also described in this section. Finally, the conclusion of the paper is given in Sec. 7.

2 Related Works

In spatial neighborhood techniques, a point or a line in the pattern is generally encoded according to the features such as color intensity, color variation, and the length or shape of a group of pixels. These techniques can be divided into three categories based on the coding strategy: nonformal codification, De-Bruijn sequences, and pseudorandom arrays.

Nonformal codification techniques, which use both one-axis[3,4] and two-axis[5,6] encodings, divide the pattern into a certain number of regions and use the information in each region to generate a different codeword. Most of these methods use random assignments or trial and error procedures to encode the pattern. The main drawback of these techniques is the ambiguity that can be incurred when identical regions appear in the pattern. The reason for this is that a mathematical theory is not followed and no constraints are applied.

Structured light patterns based on De-Bruijn sequences use the window property to define codewords in a mathematical sense.[2] The first work on a structured light system based on De-Bruijn sequences was reported by Boyer and Kak.[7] Thereafter, several researchers improved the method by using different numbers of colors, color spaces, and window sizes.[8–10] Sagawa et al.[11,12] used the De-Bruijn sequences in both the horizontal and vertical directions to achieve dense 3-D reconstructions in real time. They were able to use the proposed system on colored surfaces by using monochromic colors as the pattern stripes. Although the De-Bruijn techniques support real-time applications, there are major limitations involved owing to their one-dimensional (1-D) spatial coding method, which requires that the baseline between the camera and the projector should be nearly orthogonal to the light plane, reducing the flexibility of the system configuration.

A pseudorandom array is the two-dimensional (2-D) representation of the 1-D De-Bruijn sequence which overcomes the 1-D spatial coding problem. By definition, an M-array (perfect map) is an $r \times v$ array over an alphabet of k symbols such that every possible $m \times n$ window appears exactly once. A pseudorandom array or perfect submap is a 2-D array within which all the $m \times n$ windows are unique, but which does not necessarily contain all the possible $m \times n$ windows.[13,14]

The first study on pseudorandom arrays was conducted by MacWilliams and Sloane.[13] Their encoding method was based on arranging a larger 1-D pseudorandom sequence into a 2-D array according to a defined pattern. The difficulty involved in this method was that a very long 1-D sequence was required to create a larger resolution pseudorandom array. Griffin et al.[15] introduced another pseudorandom array encoding method that used two horizontal and vertical pseudorandom sequences. They defined the window size as 3 and considered only the 4-connectivity. The vertical $(k^2 + 2)$ and horizontal $(k^3 + 2)$ resolutions of the pattern depended on the number of symbols (k) used in the pattern. This encoding scheme had a drawback in that the width of the pattern was much greater than its height, which is neither suitable for scanning a practical target nor compatible with most light projector resolutions. In 2008, Chen et al.[16] proposed a 3-D imaging system based on Griffin's method with some modifications to overcome the long band shape of the pattern. When designing the pattern, they used seven different colors and further added a connectivity constraint and eliminated the gaps between two symbols to achieve a dense reconstruction.

Another interesting encoding method based on the brute-force approach was introduced by Morano et al.[17] This was an iterative algorithm that started from the top left corner, adding a new random codeword and checking the uniqueness of the new codeword against all the previous ones. Further, they used the Hamming distance technique to maintain a sufficient distance between two neighboring codewords. Pages et al.[18] used the same method to generate a 20×20 pseudorandom array consisting of the colors red, green, and blue with a 3×3 window property and used it as a visual servoing approach. Albitar et al.[19] also used Morano's brute-force approach to create a 27×29 monochromic pattern with a 3×3 window property and three different symbols, but instead of different colors, a filled circle, circumference, and stripe were used. Central symmetry was also considered as they aimed to integrate the pattern in an endoscopic instrument where the pattern was encoded using diffractive optical elements. To achieve dense reconstruction, Desjardins and Payeur[20] introduced a method that matches the pattern in the horizontal and vertical directions over the scene. They also used a pattern with a 53×38 resolution and different colors, which was generated according to Morano's method. Adan et al.[21] introduced a hexagonal perfect sub-map, which was different from all the other squared patterns. They also used the brute-force technique to encode the pattern, but the repetition of codewords was allowed while a substantial Hamming distance between neighbors was considered. Their system was successfully tested using moving scenes.

The main advantage of the spatial neighborhood techniques is the property that a single pattern is used, which allows their use in dynamic scene capturing. A simple hardware configuration and strong tolerance to noise in the scene are two of the other advantages of these techniques. However, the decoding stage of the neighborhood codification is more difficult since the spatial neighborhood cannot always be recovered when uncertain shadows or occlusions are present. Therefore, these techniques require an effective encoding method that helps increase the robustness of the decoding process and a better decoding method that yields accurate 3-D reconstruction results even when uncertain shadows or occlusions exist. The research presented in this paper is directed to find such an enhanced encoding and decoding method based on spatial neighborhood theories.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

3 Overview of the Proposed Method

The pseudorandom array techniques with unique patterns described in Sec. 2 have advantages over the other methods in terms of scanning speed and hardware implementation. Currently, these methods are used in commercial depth sensors, such as Microsoft Kinect, where a binary pattern is projected via infrared light. The problem pertaining to such binary patterns is that a larger window size is required to achieve a higher resolution. As a result, pattern decoding is very difficult, in particular in occluded or shadowed areas, resulting in noisy reconstructions. These reconstructions are sufficiently good for most entertainment systems but not for accurate 3-D modeling purposes.

Pseudorandom arrays encoded using a smaller window size and a smaller number of color primitives can achieve better reconstruction accuracy although they suffer from limitations when textured scenes are involved. The problem pertaining to the color pseudorandom arrays is that some pattern symbols cannot be extracted from the captured image because of uncertain occlusions, shadows, light conditions, depth variations, surface reflections, defocusing, and textures of the scene. In addition, some extracted symbols cannot be decoded when they have a missing neighbor(s), and this will eventually lead to holes in the 3-D shape reconstruction. As an initial step to solve this problem, we introduced a dual pseudorandom array encoding method, which first encodes a binary pattern with a (7×7) window property and then encodes a color pattern with a (3×3) window property on top of the binary pattern.[22] Hence, each symbol of the pattern has two codewords: a 3×3 color codeword and a 7×7 binary codeword. In Table 1, the resolution, number of symbols, and codeword size of the proposed method are compared with those of some other available pseudorandom array techniques. Only Griffin's method[15] succeeds in achieving a higher resolution than the proposed method when six or fewer symbols are used. However, as the width to height ratio is very high, Griffin's method generates a long band-like pattern that is not suitable for practical 3-D scanning.

Then we introduced a two-way decoding method that decodes a given symbol using a 3×3 color codeword and a 7×7 binary codeword even when a few neighbors are missing.[23] Here, we present a complete error-correcting 3-D scanning technique that uses the two-way encoding and decoding methods mentioned above, together with details of a hole-filling algorithm. This algorithm can be used to recover symbols that cannot be extracted from the captured image. The experimental results show that the proposed method can reconstruct the 3-D shapes of real objects more completely than the conventional methods described in Sec. 2, using a calibrated projector-camera system having a small rotation around the optical axis.

4 Dual Pseudorandom Array Encoding

The proposed pseudorandom array is an $(m \times n)$ matrix M filled with color primitives representing the alphabet $P = \{1,2,3,\ldots,p\}$, where different numbers represent different colors. As mentioned above, this pseudorandom array consists of two patterns: a binary pattern with a (7×7) window property and a color pattern with a (3×3) window property. The task of assigning color primitives to the pattern is divided into two stages: binary encoding and color encoding. In the following sections, these two tasks are described, and in Sec. 4.4, we explain the determination of the size of the matrix M and the alphabet P.

4.1 *Binary Pattern Encoding*

In the binary encoding stage, only one color primitive (red) is assigned to the pattern matrix M such that each (7×7) window becomes unique within the pattern. In other words, M consists of $(m-6) \times (n-6)$ unique binary codewords. If x_{ij} is an element in the pattern matrix M, where i is the row number and j is the column number, the binary codeword w_{ij} of this location can be defined by Eq. (1). A numerical codeword can be generated by assigning the digit 1 to the red color and 0 to all remaining positions within the window:

$$w_{ij} = \{x_{i-3,j-3}, x_{i-3,j-2}, x_{i-3,j-1}, \ldots, x_{i,j}, \ldots, x_{i+3,j+1}, x_{i+3,j+2}, x_{i+3,j+3}\}. \quad (1)$$

Table 1 Comparison between different pseudorandom array encoding methods.

Encoding method	Pattern resolution	Number of symbols	Codeword size	Special notes
Mac Williams and Sloane[13]	15×17	2	3×3 window	
Griffin et al.[15]	$(k^2+2) \times (k^3+2)$	k	4 neighbors	
Chen et al.[16]	38×212	7	4 neighbors	
Morano et al.[17]	45×45	3	3×3 window	Consider Hamming distance
Pages et al.[18]	20×20	3	3×3 window	
Albitar et al.[19]	27×29	3	3×3 window	Consider central symmetry
Desjardins and Payeur[20]	53×38	3	3×3 window	Marching patterns
Adán et al.[21]	34×29	7	6 neighbors	Use a hexagonal window
Proposed	60×45	6	3×3 window and 7×7 window	Use dual codewords

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

Two conditions are considered when assigning the color primitive to the matrix M. The first condition is that all the codewords (7×7 window) within the pattern matrix should be unique, as explained in Eq. (2). Here, W is a set that consists of all the used codewords:

$$W = \left\{ w_{ij} \middle| \begin{array}{c} w_{ij} \neq w_{kl}, (i,j) \neq (k,l) \\ 4 \leq i \leq (m-3), 4 \leq k \leq (m-3) \\ 4 \leq j \leq (n-3), 4 \leq l \leq (n-3) \end{array} \right\}. \tag{2}$$

The second condition is that there should be no eight connected color primitives within the pattern. An element x_{ij} of the matrix M can either be assigned the selected color primitive (c) or not assigned any color (b). According to this condition, if a selected element in the matrix is assigned the color primitive, then none of its eight neighbors (S) cannot be assigned the color primitive and should remain without any color. Equation (3) represents this condition:

$$M = \left\{ x_{ij} \middle| \begin{array}{c} x_{ij} = c, x_{kl} \neq c, x_{kl} \in S \\ x_{ij} = b \end{array} \right\}. \tag{3}$$

The first condition is introduced to simplify the correspondence problem of stereo vision. If each symbol in the pattern can be uniquely identified, then it will be easy to find the correspondences between the two views, the projected pattern and the captured image. The second condition is introduced to simplify the pattern segmentation process. If there are connected color symbols in a dense binary pattern, it is difficult to identify these symbols separately from the captured image in the segmentation process.

4.2 *Color Pattern Encoding*

After the binary pattern has been encoded, the color pattern is encoded on top of the binary pattern. Here, all the elements of the pattern matrix that do not have any color assignment in the binary encoding process are assigned a new color primitive from the alphabet P other than the color used in the binary encoding. Then the color codeword w'_{ij} of a selected element x_{ij} from the matrix M can be defined by the numerical representation of the color primitives inside the (3×3) window centered on the selected element, as shown in Eq. (4). The entire matrix M consists of a total of $(m-2) \times (n-2)$ color codewords:

$$w'_{ij} = \{x_{i-1,j-1}, x_{i-1,j}, x_{i-1,j+1}, x_{i,j-1}, x_{i,j}, x_{i,j+1}, x_{i+1,j-1}, x_{i+1,j}, x_{i+1,j+1}\}. \tag{4}$$

The uniqueness condition explained in Eq. (5) and the not connected to the same color symbols constraint explained in Eq. (6) are applied to the color encoding as to the binary encoding. After color encoding is completed, every element x_{ij} in the matrix M is assigned a color primitive. Furthermore, the density of the pattern increases, as no gaps exist between pattern symbols. Therefore, it is important to apply the connectivity constraint to differentiate each symbol from its eight neighbors in the pattern segmentation process:

$$W' = \left\{ w'_{ij} \middle| \begin{array}{c} w'_{ij} \neq w'_{kl}, (i,j) \neq (k,l) \\ 2 \leq i \leq (m-1), 2 \leq k \leq (m-1) \\ 2 \leq j \leq (n-1), 2 \leq l \leq (n-1) \end{array} \right\} \tag{5}$$

$$M = \left\{ x_{ij} \middle| \begin{array}{c} x_{ij} \neq x_{kl}, (i,j) \neq (k,l) \\ x_{kl} \in S \\ 2 \leq i \leq (m-1) \\ 2 \leq j \leq (n-1) \end{array} \right\}. \tag{6}$$

4.3 *Encoding Algorithm*

An algorithm similar to that introduced by Morano et al.[17] is used for both binary and color pattern codifications. For example, to generate a pattern with a (3×3) window property, Morano's algorithm starts by seeding the top left (3×3) window with random letter assignments taken from the alphabet P. Then consecutive (3×1) columns with random letter assignments are added to the right of the initial window, maintaining the uniqueness of each (3×3) window. Next, random (1×3) rows are similarly added beneath the initial window. Then both the horizontal and vertical processes are repeated by incrementing the starting position by one in both directions until the whole pattern is filled. In the final step, the algorithm adds only a single point to the pattern and checks the constraints. Whenever the process reaches a position where no possible symbol can be placed, the entire pattern is cleared, and the process starts again using another initial window.

The same algorithm is used to encode the proposed binary pattern with the (7×7) window property, along with an added constraint that checks the 8-connectivity. The alphabet used in the binary encoding process contains only one symbol ($P = \{1\}$) and is represented by the color red on the pattern image. When numerically representing the codewords, the digit zero is used to symbolize the elements to which red is not assigned. In addition to the uniqueness of the (7×7) window, the algorithm checks also whether there exist any connected red symbols within the window. Figure 1 shows an example of encoding a binary pattern having a 10×9 resolution.

To encode the color pattern on top of the binary pattern, first the encoding algorithm loads the binary pattern and then replaces all the zeroes with a color primitive taken from the alphabet $P = \{2,3,4,5,6\}$ (green, blue, cyan, magenta, and yellow). The same algorithm is used to assign the color primitives with a (3×3) window property. Figure 2 shows an example of color encoding that uses the binary pattern generated in Fig. 1.

4.4 *Pattern Resolution*

The maximum resolution of a pseudorandom pattern depends on the window size and the number of symbols. The resolution can be increased when the window size or number of used symbols is increased. However, the complexity of the decoding algorithm will be increased when the window size and number of symbols increase. To maintain the balance between the maximum resolution and the computational complexity, the proposed method uses the (7×7) window property for the binary pattern, the (3×3) window property for the color pattern, and six different

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

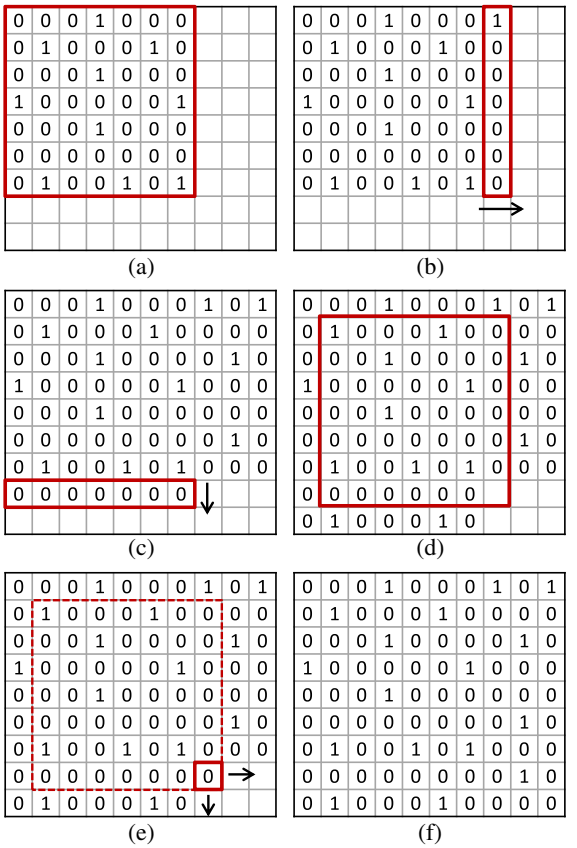


Fig. 1 The process of binary pattern encoding: (a) Step 1: seeding the initial window with random letter assignments; (b) Step 2: assigning (7×1) random columns to the right of the initial window; (c) Step 3: assigning (1×7) random rows beneath the initial window; (d) Step 4: incrementing the starting position by one row and column; (e) randomly adding a 0 or a 1 to complete the windows and to continue the process horizontally and vertically; and (f) completed binary pattern.

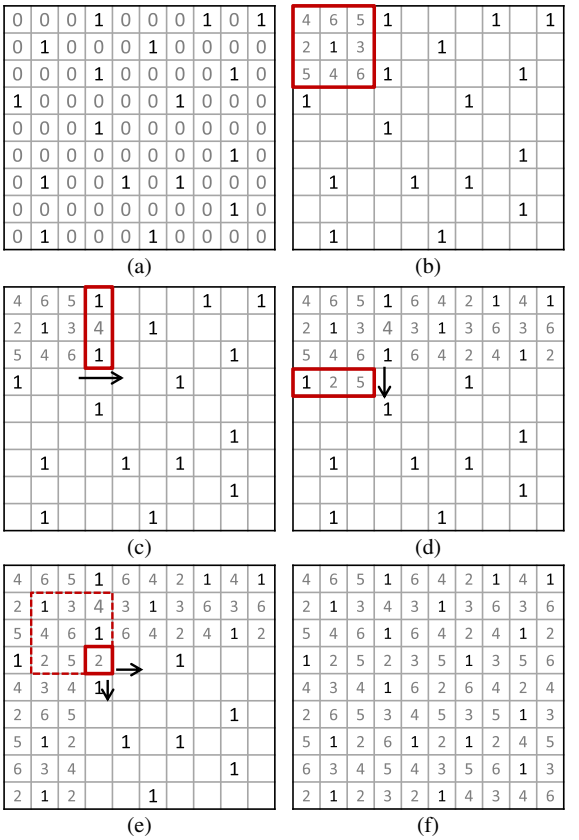


Fig. 2 The process of color pattern encoding: (a) Step 1: loading the binary pattern and removing all the zeros; (b) Step 2: assigning random color primitives to the initial window; (c) Step 3: assigning (3×1) random columns to the right of the initial window; (d) Step 3: assigning (1×3) random rows beneath the initial window; (e) Step 4: incrementing the starting position by one row and column and continue; and (f) completed dual pseudorandom array.

colors (red, green, blue, cyan, magenta, and yellow) to represent pattern symbols. Practical digital projectors have resolutions with a 4:3 or 16:9 width to height ratio. Therefore, it is preferred to generate a pattern that follows these resolutions.

When we adhered to all these conditions, (60×45) was the maximum pattern resolution we could achieve. The procedure for obtaining the maximum resolution is further discussed in Sec. 6.2.1. Each symbol in the pattern is represented by a 10×10 pixels square, where no gaps exist between symbols. Therefore, the final pattern is a 600×450 pixels color image, as shown in Fig. 3.

5 Dual Pseudorandom Array Decoding

The determination of the correspondences between the projected pattern and the captured image is one of the most important steps in 3-D reconstruction. In the pattern decoding stage, the codeword of each pattern symbol in the captured image should be accurately determined to solve the correspondences. A dual pseudorandom pattern that encodes two unique codewords for each symbol is introduced in the proposed encoding method. In this section, a two-way decoding method and a hole-filling algorithm to achieve better reconstructions with these two codewords are introduced.

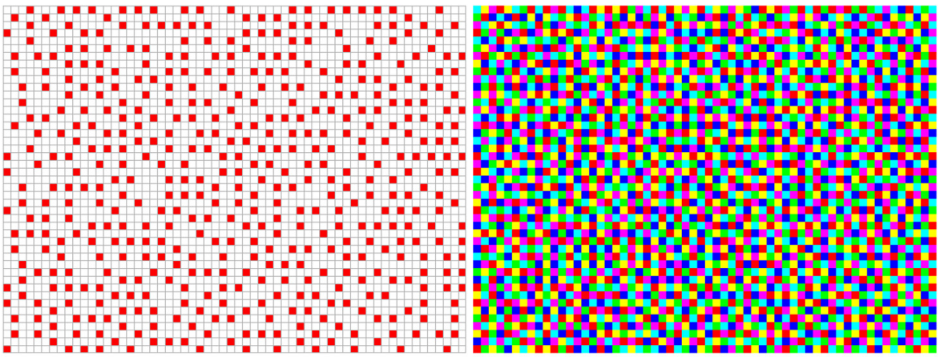
The flowchart in Fig. 4 shows the coordination of the four main stages of the proposed decoding process, square grid point extraction using color segmentation, (3×3) color decoding, (7×7) binary decoding, and hole filling.

5.1 *Color Segmentation*

In this stage, all the square grid points that represent the centers of small colored squares are extracted from the captured image. First, the captured image is converted to the hue, saturation, and value (HSV) color space, as in the example shown in Fig. 5. Then the value component is used to create a mask by applying a simple binary thresholding to identify the projection area of the pattern separately from the whole captured image [Fig. 5(e)].

The hue component is used to separately identify the six colors, as it is invariant to the brightness level of the image and depends only on the color value. The histogram of the hue component clearly has six peaks which are related to the six different colors. Five threshold values can be defined by analyzing the histogram, and these values can be used to segment the image into six binary images that can be related to a particular color. Figure 6(a) shows an example of a binary image that corresponds to the red color extracted from Fig. 5(a) using the above method. Morphological filters are applied to remove noise in these six binary images. The erosion filter defined in Eq. (7) is first applied to shrink all

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms



(a) (b)

Fig. 3 Proposed pattern: (a) binary pattern encoded with (7×7) window property; and (b) final color pattern encoded on top of the binary pattern with (3×3) window property.

the segments in the binary image and then the dilation filter defined in Eq. (8) is used to expand the segments smoothly. Figure 6(b) shows the binary image after applying morphological filters.

$$\mathrm{dst}(x,y) = \min_{-d/2 \le x',y' \le d/2} \mathrm{src}(x+x', y+y') \tag{7}$$

$$\mathrm{dst}(x,y) = \max_{-d/2 \le x',y' \le d/2} \mathrm{src}(x+x', y+y'). \tag{8}$$

Since every symbol is a different color from its eight neighbors, a contour detection algorithm can be separately applied to each binary image to locate the color symbols in the captured image. The size of each contour is measured and compared with the average size of all contours. By using this comparison, very large or small contours that are not related to any pattern symbol can be removed. Then all the square grid points can be extracted by finding the mass center $(\bar{x}, \bar{y})$ of the contours using Eq. (9), where $m_{i,j}$ is a spatial moment of a contour which can be computed by Eq. (10). Detected contours and extracted red square grid points are shown in Figs. 6(c) and 6(d), respectively.

$$\bar{x} = \frac{m_{1,0}}{m_{0,0}}, \qquad \bar{y} = \frac{m_{0,1}}{m_{0,0}} \tag{9}$$

$$m_{i,j} = \sum_{x,y} \mathrm{contour}(x,y).x^i.y^j. \tag{10}$$

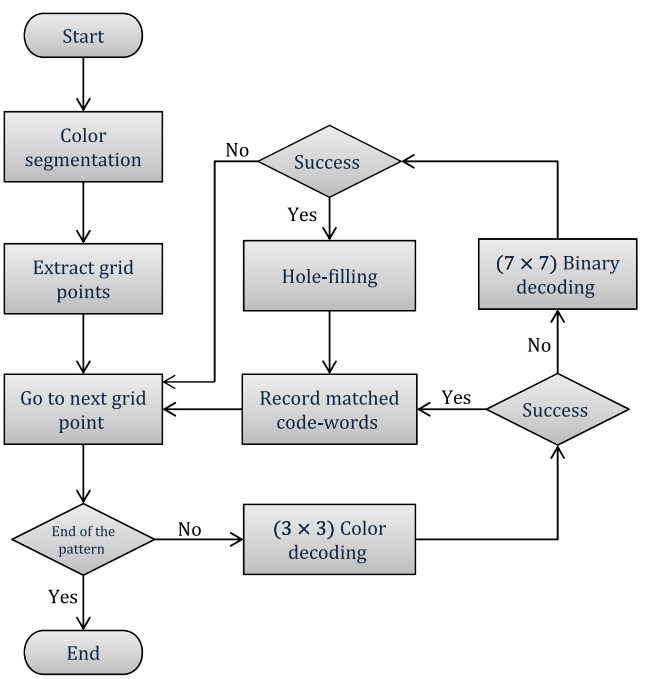


Fig. 4 Flowchart of the pattern decoding process.

5.2 *Color Decoding*

After all the square grid points are extracted from the six binary images, they are combined to make a grid network by connecting four neighbors (Fig. 7). The city block distance defined in Eq. (11) is used to find the distance between two grid points $(p_{x,y}, q_{i,j})$ when finding the four nearest neighbors using the k-nearest neighbors algorithm. This distance is compared with the average distance between two grid points around the neighborhood to decide whether they are the actual four adjacent neighbors. If a grid point is connected with incorrect neighbors because of occlusion or segmentation errors, then an incorrect codeword will be generated for that grid point. Some of these connection errors can be corrected in the binary decoding and outlier removal processes. The remaining grid points with codeword errors are eliminated from the 3-D reconstruction:

$$\mathrm{dist}(p_{x,y}, q_{i,j}) = |x-i| + |y-j|, \qquad (x,y) \neq (i,j). \tag{11}$$

The decoding algorithm examines each grid point of the generated network and attempts to decode the (3×3) color codeword. For each grid point, a (3×3) network centered at the chosen grid point is extracted from the whole network using a flood search algorithm. As explained in Fig. 7, the flood search algorithm first chooses the middle grid point and finds the neighbors that are connected to it in the network. Then it takes each of these neighbors and finds the grid points that are connected to them. It continues likewise until it fills the whole matrix or no more connected neighbors exist within the required area. For color decoding, only a (3×3) grid network has to be extracted, whereas for binary decoding a (7×7) network must be extracted. Some missing connections can occur when generating the grid network using the segmentation results. Using this flood search algorithm, it is possible

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

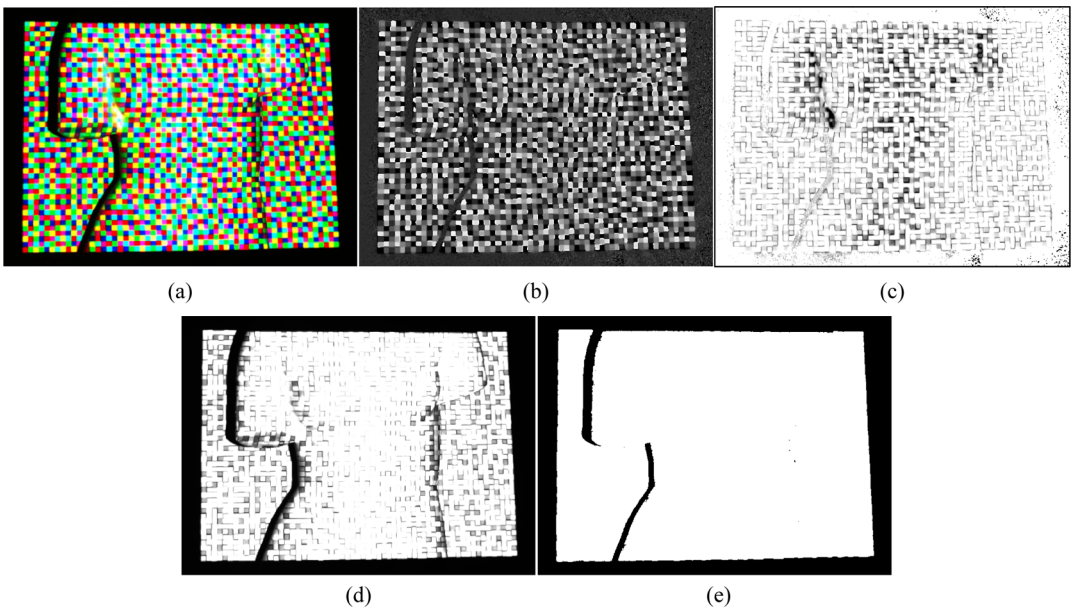


Fig. 5 Conversion to the HSV color space and creation of a mask for identifying the pattern projected area: (a) captured image; (b) hue component; (c) saturation component; (d) value component; and (e) mask created using the value component.

to correct these missing connections in the grid network. After the (3×3) network has been extracted, corresponding numerical values are assigned to each of the color symbols and the corresponding codeword is generated. Then this unique codeword is found in the projected pattern and the correspondences are solved to create the 3-D model.

5.3 *Binary Decoding*

If there is a missing square grid point within the (3×3) network, color decoding cannot be continued because of the uncompleted codeword. When this occurs, the proposed decoding algorithm is extended to include the binary decoding method, which considers only the red square grid points.

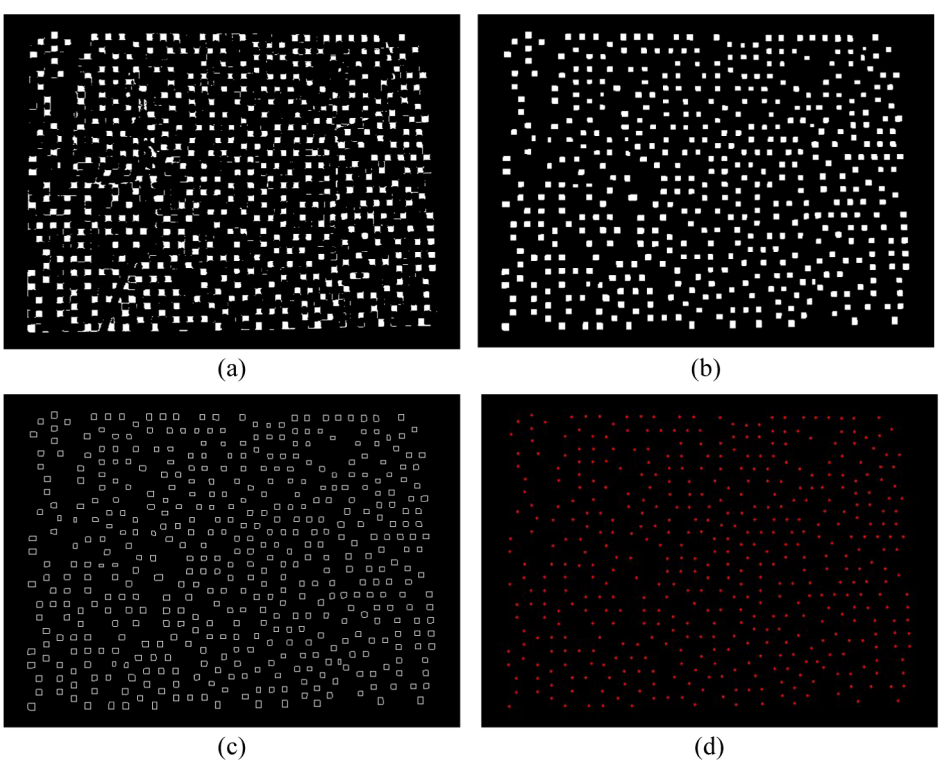


Fig. 6 Square grid points extraction process: (a) binary image corresponds to the red color which is segmented from the hue image shown in Fig. 5(b); (b) after applying the morphological filters; (c) contour detection; and (d) extracted red square grid points.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

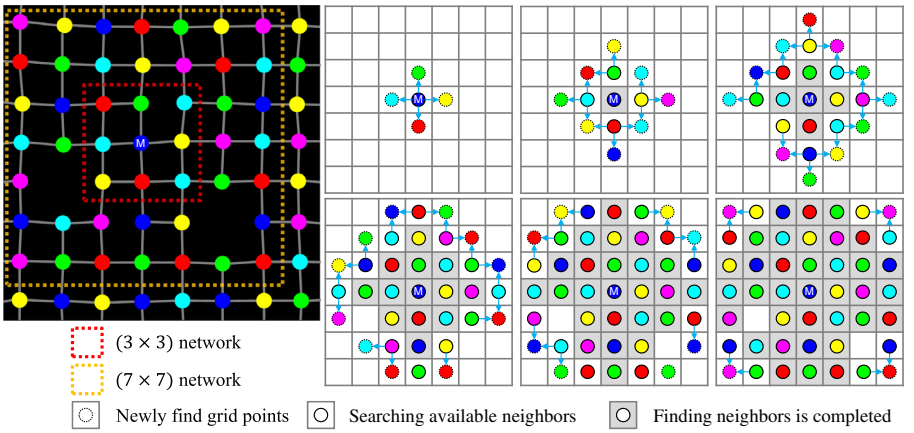


Fig. 7 Flood search algorithm to extract (3 × 3) or (7 × 7) network for decoding the codeword. The left figure shows the grid network generated using the segmentation results. The remaining figures show the steps of the flood search algorithm for extracting a (7 × 7) network.

Then the decoding can be handled even when a few missing neighbors exist.

The same flood search algorithm explained in Fig. 7 is used to extract the (7 × 7) network, which is centered on the square grid point for which the binary decoding is required. Then all the red square grid points are located on the network and the binary codeword is generated by assigning the numerical value 1 to the color red and 0 to all the other colors and holes. This binary codeword may not be the exact codeword of the chosen square grid point because some missing red grid points can exist in the extracted network. Therefore, this generated codeword is passed through a refinement process before it is used to solve the correspondences. First, codewords that are similar to the generated codeword are chosen from the used codewords list (all the binary codewords used in the projected pattern). The Hamming distance technique explained in Eq. (12) is used to measure the similarity between two codewords. In this equation, d^{HAD} represents the Hamming distance between the codewords i and j, whereas k represents the index of the respective character reading y out of an n total number of characters. The Hamming distance represents the number of mismatches between the characters of the codewords paired with k:

$$d^{\mathrm{HAD}}(i,j) = \sum_{k=0}^{n-1} [y_{i,k} \neq y_{j,k}]. \tag{12}$$

If the Hamming distance $d^{\mathrm{HAD}}(i,j)$ between the generated codeword i and the codeword j chosen from the used codewords list is less than four $[d^{\mathrm{HAD}}(i,j) < 4]$, then the binary codeword j and its corresponding (7 × 7) color codeword are both added to the list of similar codewords. When all the similar codewords have been found, the exact matching codeword is selected by performing a color comparison. This refinement process compares the colors of the available square grid points on the extracted (7 × 7) network with the corresponding color values of the color codewords in the similar codewords list. If all the available grid points in the extracted network correctly match a color codeword, its binary codeword is chosen as the exact machine codeword of the middle grid point. Figure 8 shows an example of the binary decoding process. In this example, only the first codeword in the similar codewords list passes the color comparison, hence, it is used as the exact matching codeword.

5.4 *Hole-Filling Algorithm*

For various reasons, such as depth variation, external light conditions, and object surface characteristics, it is difficult to extract every visible projected pattern symbol from the captured image. Figure 9 shows some examples of such situations where missing symbols can occur. Then some of the extracted pattern symbols cannot be decoded using conventional color decoding methods because their missing neighbors. The proposed hole-filling algorithm with a few refinement steps, which are explained in Fig. 10, is introduced to enhance the final reconstruction by reacquiring these missing symbols using the available neighbor information.

In the previous section, we explained the decoding of a given grid point even when it has some missing neighbors (grid points). These decoding results are used to find the required information such as the color, corresponding point, and image coordinates of the missing grid points. First, the corresponding point of the hole is found using the information of its position relative to the midpoint of the extracted (7 × 7) grid network. The correspondence of the midpoint is already known from the decoding results. The first refinement step is executed after finding the correspondence of the hole. We keep the records of the decoding status of each symbol in the pattern and check whether the found correspondence of the hole is already a decoded symbol. If so, the reacquiring process of that hole is discontinued and the algorithm proceeds to the next hole in the network; otherwise, it is advanced to the image coordinates estimation process.

The affine transformation technique is employed to estimate the image coordinates of the hole using the image coordinates of the available neighbors and their correspondences, as in Fig. 11. Here, we assume that the area of the (7 × 7) pattern window projected on the real object is nearly a plane and no depth variances exist within the area. To support this assumption, we made the pattern symbols as small as possible. In addition, the projection area of a (7 × 7) window from a 1 m distance is about 40 × 40 mm^2, according to

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

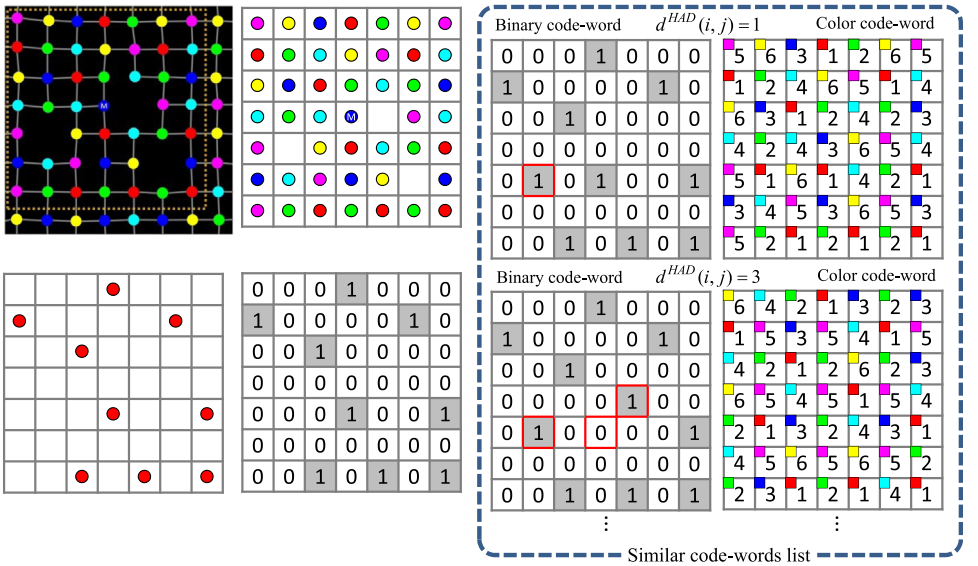


Fig. 8 Binary decoding process: (a) (7 × 7) grid network for binary decoding; (b) (7 × 7) window extracted using the flood search algorithm; (c) locate the red color grid points; (d) generate the binary codeword using the red color grid points; and (e) similar codewords list.

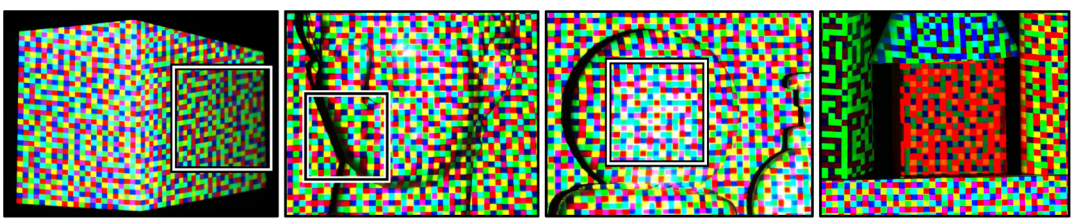


Fig. 9 Example situations where missing symbols can occur; (a) depth variation; (b) shadows and occlusions; (c) surface reflection; and (d) surface texture.

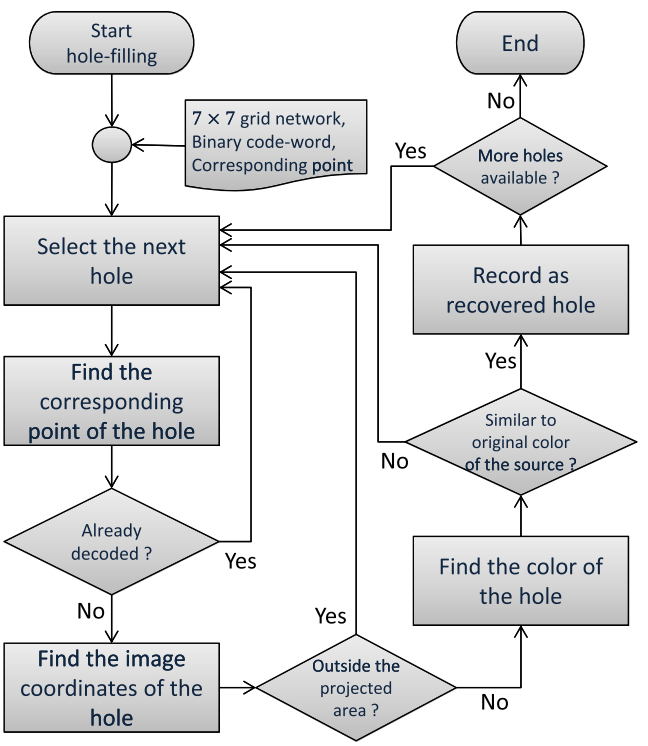


Fig. 10 Flowchart of the hole-filling process.

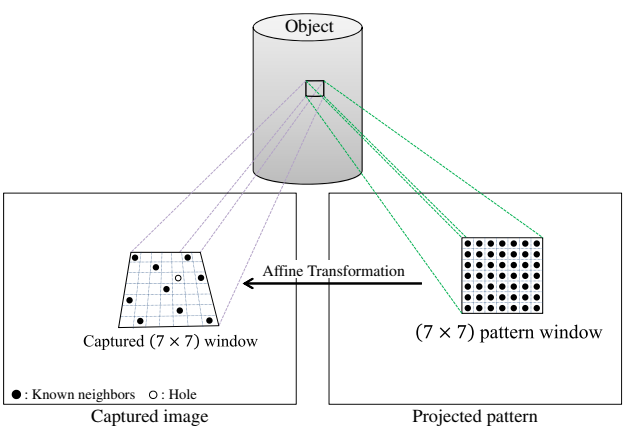


Fig. 11 Application of the affine transformation to find the image coordinates of a hole using neighbor grid points.

the projector specifications (the projection area of a 1280 × 800 image from a distance of 0.9 m is 646.2 × 403.9 mm^2) and the experimental setup, in which the 600 × 450 resolution pattern image is projected from a 1 m distance. Considering the size of a single-grid point (6 × 6 mm^2), the error of the affine transformation as compared with the projective transformation is negligible.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

At least three known neighbors are required to find the image coordinates of the hole using the affine transformation. Correspondences of these known neighbors can be used to derive the transformation matrix. Image coordinates of a hole in the camera view can be found using known correspondences and the transformation matrix. This method, with a slight modification, was used in our study to directly find the image coordinates of the hole without finding the transformation matrix.

The refinement stage of finding the holes that are generated outside the projection area of the pattern is the next step in the hole-filling process. This refinement stage is very important when the hole-filling algorithm is applied in the vicinity of the occluded areas, as shown in Fig. 12(a). In this figure, the first two cells in the first column of the (7×7) window are considered as holes when generating the (7×7) grid. In fact, they are not holes and no pattern symbols are projected in that area because of occlusion. Therefore, in this refinement stage, the generated image coordinates and the mask created in the pattern segmentation process are used to check whether the hole is outside the projection area of the pattern [Fig. 12(b)].

The next step in the hole-filling process is finding the associate colors of the newly filled holes. As a result of the binary decoding process, we have the (7×7) binary codeword and the (7×7) color codeword of the window. The color values of all the missing symbols within the window can be found by fitting the color codeword to the window, as shown in Fig. 13. The third refinement step is checking whether the associated color of the filled holes is similar to the color of the same position on the original image. A color distance measurement technique is used to measure the similarity; if the color is completely different, that newly filled hole is removed. If a missing symbol within a (7×7) window passes all three refinement stages and completes the hole-filling process, it is recorded as a filled hole and sent to the 3-D reconstruction process.

Three-dimensional interpolation is another hole-filling technique that can be used to find missing symbols after the initial reconstruction is complete, whereas the proposed method finds missing symbols in 2-D space first. As shown in Fig. 14, the proposed method has the advantage of finding more accurate 3-D information, as it can find the correspondences in 2-D space and apply the triangulation.

5.5 *Outlier Removal*

Erroneous connections, which cause false correspondences in the decoding process, can occur as a result of various occlusions that appear in the scene and segmentation errors. Three different methods are followed during the decoding process to handle incorrect connections.

When a grid point is decoded successfully, the correspondences of the grid point and its eight neighbors can be found. Before decoding is applied to a particular grid point, the proposed method checks whether previously found correspondences exist. If so, 3-D reconstruction is still possible, even when erroneous connections exist at the grid

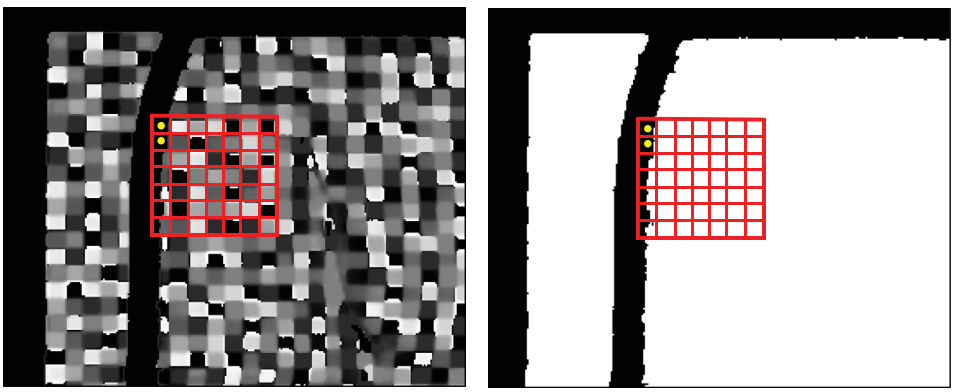


Fig. 12 Application of the hole-filling algorithm in an occluded area. The generated image coordinates in (a) are checked using the mask as in (b).

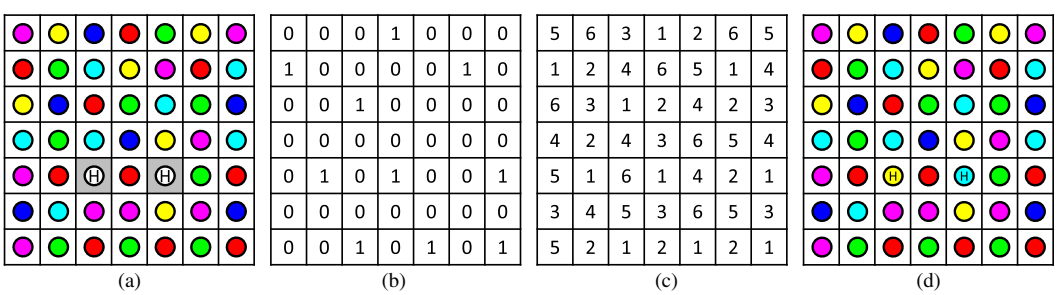


Fig. 13 Process of finding the colors of holes: (a) a (7×7) window with two holes; (b) generated binary codeword of the window; (c) corresponding color codeword of the window; and (d) filled holes with colors using the color codeword.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

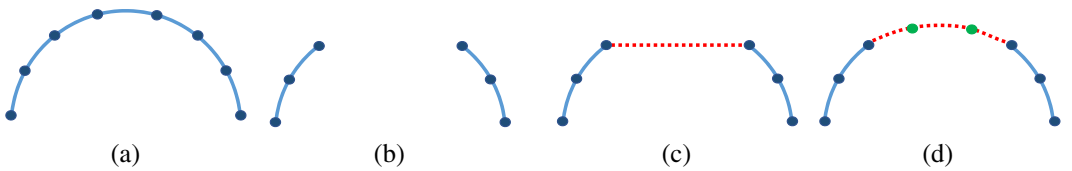


Fig. 14 Difference between three-dimensional (3-D) interpolation and the proposed hole-filling method: (a) original model; (b) model with holes; (c) after completing the model using 3-D interpolation techniques; and (d) after completing the model using proposed hole-filling method.

point. The flood search algorithm is used to correct the erroneous connections, as it checks whether the four neighbors of a selected grid point are the same as the connected grid points' neighbors. If a grid point is connected with incorrect neighbors, an incorrect 3×3 codeword, which may not be included in the code base (all used codewords), is generated for that grid point. If the codeword is not included in the code base, this assures us that the grid point has incorrect connections and can be eliminated from further processing. If incorrect connections still exist, false correspondences could be created. The following outlier removal technique is applied to remove these false correspondences.

The fundamental matrix, F, which relates corresponding points in stereo images, is used in our proposed outlier removal process. In epipolar geometry, when x and x' are corresponding points in a stereo image pair with homogeneous image coordinates, Fx depicts an epipolar line on which the corresponding point x' on the other image must lie. This means that Eq. (13) must hold for all pairs of corresponding points:

$$x'^{T} F x = 0. \tag{13}$$

Intrinsic and extrinsic calibration parameters of the projector-camera system are used to find the fundamental matrix F. In practical situations, even for correct correspondences this equation does not always yield zero, but rather a very small value. Therefore, upper and lower thresholds are defined to check whether a given correspondence is an inlier or outlier.

6 Experimental Results and Discussion

To implement the coding and decoding ideas in a practical vision system, some other techniques and algorithms, such as system calibration, image processing tasks, 3-D reconstruction, and 3-D representation, have to be considered in the automation of the modeling process.

6.1 *Experimental Setup*

The proposed method was implemented in a vision system comprising a Point Grey GS2-FW-14S5C-C CCD camera with a 16-mm lens, which provides images of dimensions 1290×960, and an Optoma ML500 LED projector with a 3000:1 contrast ratio. According to the projector specifications, the projected area of an 1280×800 image from a distance of 0.9 m is 646.2×403.9 mm^2. Since the resolution of the projected pattern is 600×450, the projection area of the pattern from a 1 m distance is nearly 336×252 mm^2. The baseline between the camera and the projector was about 12 cm and the distance between the scene and the projector-camera system was about 1 m (Fig. 15). All computations were performed on a normal PC with a 2.67 GHz Core i5 CPU and 4 GB RAM. The OpenCV library was used for image processing tasks and the Point Cloud Library for 3-D representation tasks.

The calibration of the projector-camera system is one of the most challenging tasks in 3-D reconstruction using structured light systems. Several methods based on different techniques for calibrating a projector-camera stereo system have been introduced thus far.[24–26] A method based on planar homography introduced in Ref. 27 was used in our system to calibrate the projector-camera system. This method introduced a technique for finding the homography between the camera and the projector to generate the projector image of a planar calibration pattern from the camera image. Then Zhang's[28] camera calibration method was applied to the camera and projector images of the pattern board to calibrate the system.

Figure 16 shows the coordinate systems of our projector-camera system. The projection matrices of the camera ($\bar{P}_c$) and the projector ($\bar{P}_p$) can be derived using the calibration parameters of the system. Given two corresponding points (u, v) in camera coordinates and (i, j) in projector coordinates, the 3-D coordinate $w = (x, y, z)$ is calculated by applying the triangulation.[29]

6.2 *Results and Discussion*

Several experiments were conducted to test the performance of both the encoding and decoding methods along with an accuracy analysis of the reconstruction results.

6.2.1 *Results of pattern encoding*

The first experiment was used to find a better pattern size for the projection. The size of the pattern depends mainly on the

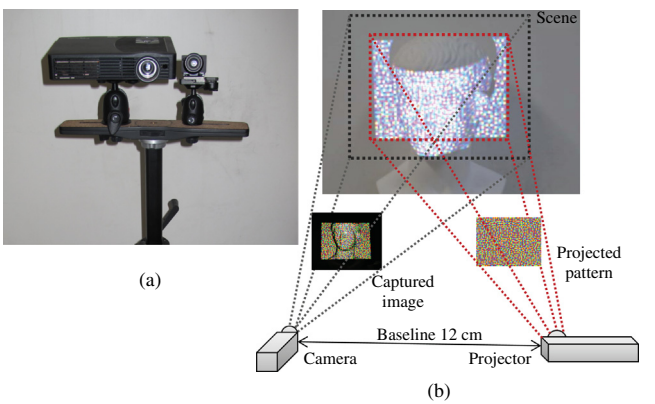


Fig. 15 Experimental setup: (a) camera and projector used in the experiment; and (b) setup of the camera and the projector in front of the scene.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

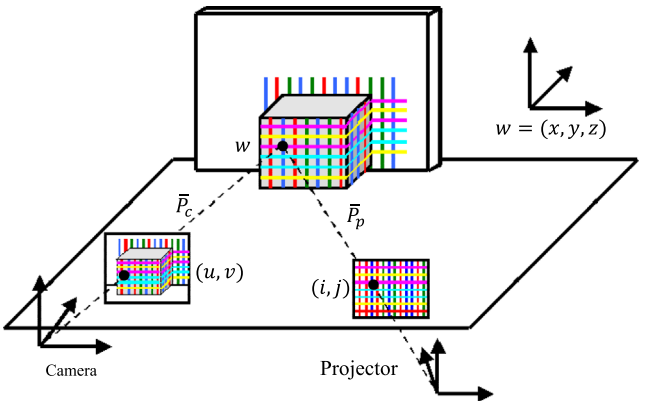


Fig. 16 Coordinate system of the proposed structured light system with a camera and a projector.

window size of the binary pattern and the number of colors used in the color pattern. The window size of the binary pattern should not only be larger than the window size of the color pattern, but should also be sufficiently small to allow decoding in a relatively short time. Considering these facts, a (7×7) window size was selected for the binary pattern and a (3×3) window size for the color pattern. Table 2 shows the results of the experiment conducted to find the largest possible resolution of a pattern with a (7×7) binary window.

The first row of the table represents the number of total codewords needed to fill the pattern using the given resolution. The last row presents the number of iterations that successfully find the full pattern out of 1000 trials. The value in the third row will be 100% if the full pattern is generated in any trial; otherwise, this row presents the maximum percentage filled by the algorithm. The average of the filling percentage of the full pattern in all 1000 trials is presented in the second row. Resolutions are chosen such that they adhere to the resolutions of the practical digital projectors. The result gives (80×60) as the largest possible resolution with (7×7) binary encoding. However, when the constraints [(3×3) window size and the use of only six colors)] are applied to the color encoding, the highest possible resolution for the pattern is (60×45).

6.2.2 *Comparison of decoding results*

In the second stage of the experiment, the proposed decoding method was compared with three other decoding methods; the results are shown in Table 3. In this comparison, conventional (3×3) color decoding and (7×7) color decoding methods are used to show the advantages of the proposed hole-filling method. A method consisting of (7×7) color decoding and hole filling is used to compare the advantages of the proposed dual-decoding method. Each row of the table represents a different method and gives the number of 3-D points reconstructed and the time consumed in the decoding process. The final row represents the proposed method. Figure 17 shows the square grid point extraction results of Scene 4 and the generated grid networks connecting the neighbors before and after applying the proposed dual decoding method. In this example, there are many missing symbols in the middle of the objects resulting from slight reflections; the proposed method can fill out these missing symbols to some extent. Figure 18 shows the reconstruction results of Scene 2 using these four different decoding methods. This result clearly proves that the two methods that use the hole-filling technique achieve more complete reconstructions than the other two methods. The comparison of the two methods that use the hole-filling algorithm showed that the (7×7) color decoding method consumed more time than the proposed dual decoding method. The reason is that the proposed dual decoding method is extended to the more time-consuming (7×7) binary decoding only if the less time-consuming (3×3) color decoding cannot be used. This comparison proves that the proposed method is suitable

Table 2 Binary encoding with (7×7) window property.

Resolution	(20×20)	(40×30)	(45×45)	(60×45)	(80×60)	(100×100)
Total number of codewords	196	816	1521	2106	3996	8836
Average (%)	99.4	91.9	75.2	62.9	36.3	17.1
Maximum completed (%)	100	100	100	100	100	50.1
Number of completed iterations	982	763	390	173	7	0

Table 3 Comparison of different decoding methods.

Decoding method	Scene 1		Scene 2		Scene 3		Scene 4		Scene 5	
3×3 color decoding	2477	0.32s	2419	0.31s	2578	0.32s	2204	0.29s	2470	0.32s
7×7 color decoding	2246	0.35s	2152	0.33s	2352	0.34s	1486	0.31s	2089	0.34s
7×7 color decoding with hole filling	2590	2.40s	2553	2.39s	2652	2.31s	2448	2.18s	2616	2.36s
Dual decoding with hole filling	2578	0.74s	2524	0.74s	2632	0.72s	2425	1.21s	2600	0.77s

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

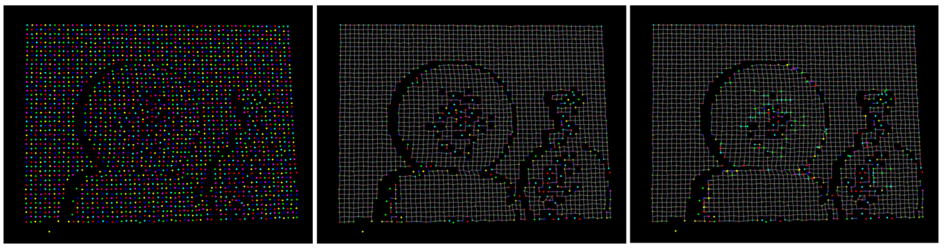


Fig. 17 Grid network representation of the decoding results: (a) extracted square grid points in the segmentation process; (b) grid network generated by connecting the neighbors for decoding; and (c) grid network after the hole-filling process.

for one-shot active 3-D scanning devices for the purpose of achieving better reconstructions.

6.2.3 *Reconstruction error analysis*

The accuracy of the reconstruction is measured by comparing a reconstructed model with its ground truth (made using a NextEngine Desktop 3D laser scanner). MeshLab is used to register (align) the two point clouds and the registration error is used as a measurement of accuracy. Figure 19 shows the 3-D registration result of a human face model, which is nearly $45 \times 30 \times 25$ cm^3 in size and 1m away from the scanner. Here, the registration error is 1.29 mm (see Table 4), where the registration is performed by first aligning the two-point clouds by selecting the corresponding points manually, and then executing the iterative closest point algorithm. Since it is very difficult to judge the quality of the reconstruction based on a color-coded presentation, Fig. 20 demonstrates the 3-D reconstruction of Scene 2 from different viewpoints for a better visualization.

6.2.4 *Reconstruction results of different scenes*

In the final experiment, the robustness of the proposed algorithm was checked by reconstructing five different scenes; the results are shown in Fig. 21. All the objects in these five scenes were 1 m away from the camera-projector system and varied in size from $2 \times 5 \times 10$ cm^3 to $25 \times 30 \times 50$ cm^3. The first column of the figure shows the five scenes, the second column shows the point cloud reconstructed using

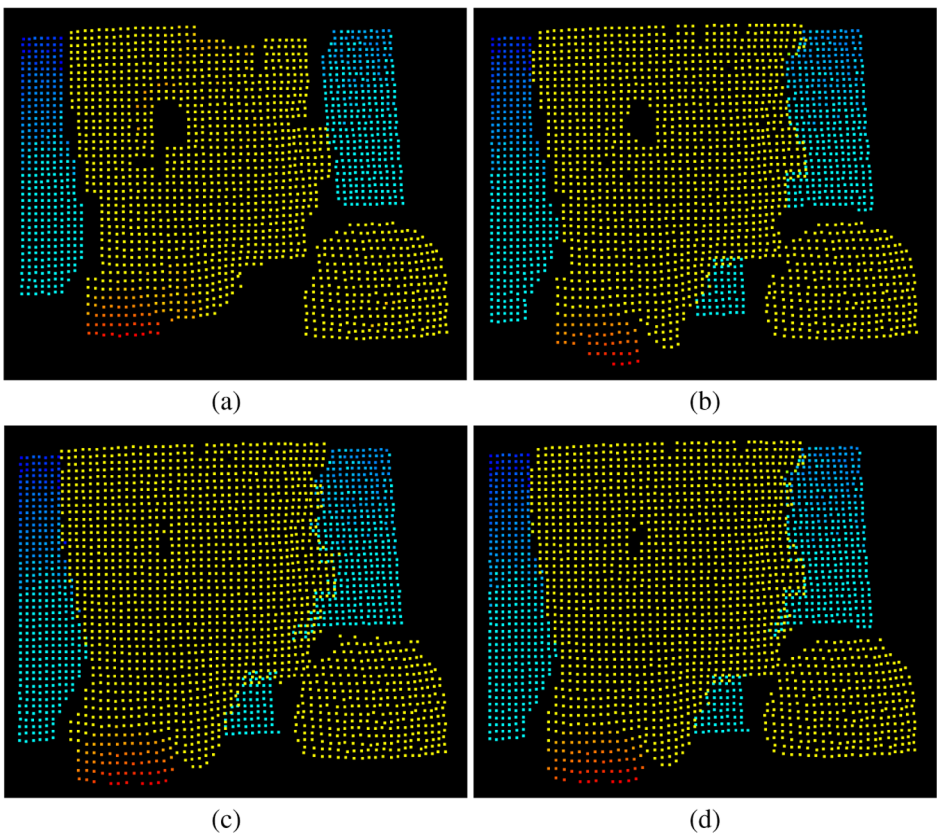
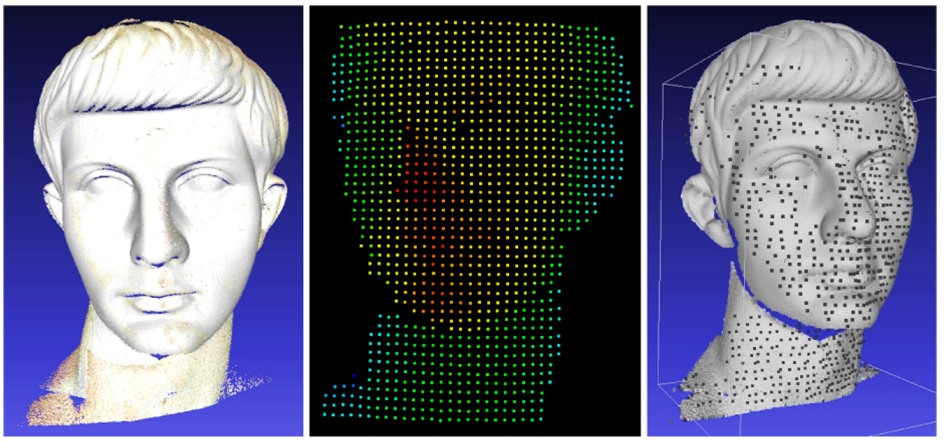


Fig. 18 Comparison of the reconstruction results of the four decoding methods: (a) (7×7) color decoding; (b) (3×3) color decoding; (c) (7×7) color decoding with hole filling; and (d) proposed dual decoding method with hole filling.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms



(a) (b) (c)

Fig. 19 Measuring the reconstruction accuracy of the proposed method: (a) ground truth; (b) proposed method; and (c) aligning results.

Table 4 Reconstruction accuracy analysis.

# of reconstructed points	Object size	Registration error
2578	$45 \times 30 \times 25$ cm^3	1.29 mm

conventional (3×3) color decoding, and the third column shows the point cloud reconstructed using the proposed dual decoding method with hole filling. In the segmentation process, most of the symbols in areas that have many shadows, illuminations, or reflections cannot be extracted. Therefore, the results show a very sparse reconstruction in those areas when the conventional color decoding is used. However, the reconstruction results are much better in those areas when the proposed method is used.

Further, these results are compared numerically in Table 5. The total number of symbols in the projected pattern was always 2700, as the (60×45) pattern was used for all the experiments. The second row of the table shows the number of square grid points extracted from the captured image in the segmentation process and its percentage of the total number of projected symbols. The third row shows the number of 3-D points reconstructed using the conventional (3×3) color decoding and its percentage of the total number of projected symbols. In most cases, these percentages are lower than the percentages of the segmentation results,

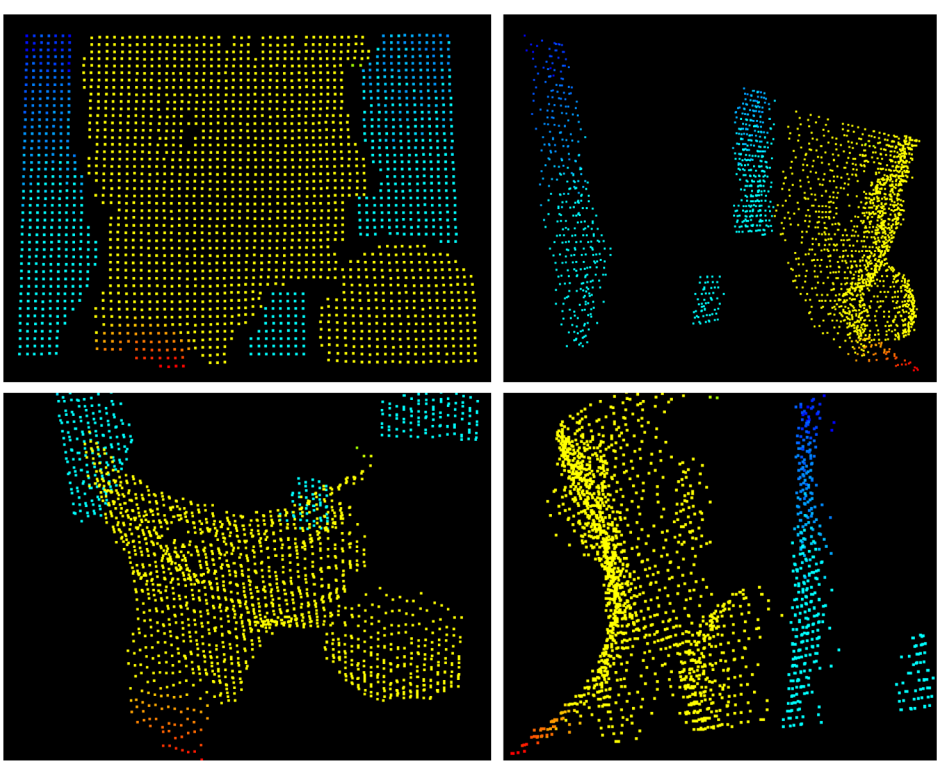


Fig. 20 Reconstruction results of Scene 2 in different viewpoints.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

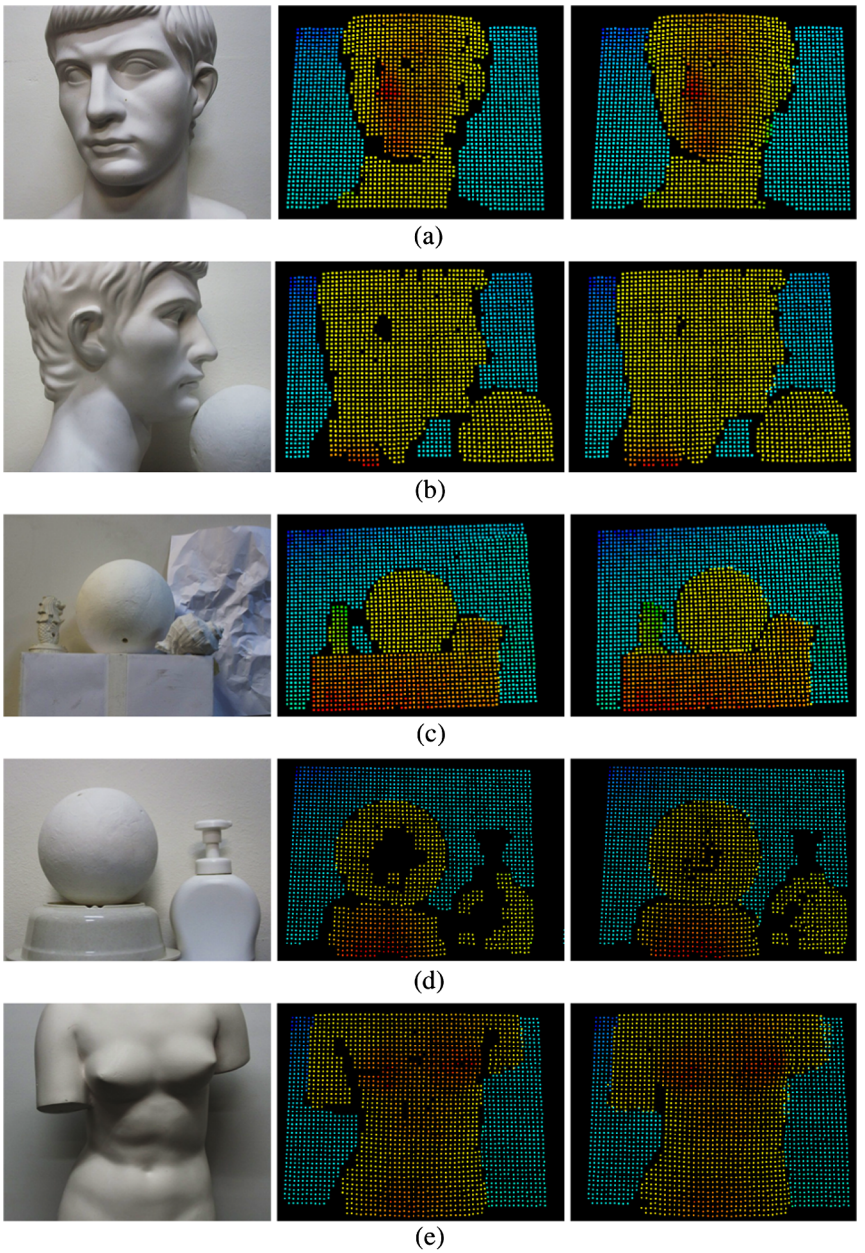


Fig. 21 Three-dimensional reconstruction results of the five scenes. The first column shows the five scenes, the second column shows the 3-D point clouds generated using conventional 3×3 color decoding, and the last column shows the 3-D point clouds generated using the proposed dual decoding method with hole filling: (a) Scene 1: human face model front view; (b) Scene 2: human face model side view; (c) Scene 3: multiple objects I; (d) Scene 4: multiple objects II; and (e) Scene 5: Venus statue front view.

Table 5 Numerical comparison of the reconstruction results.

Experiment model		Scene 1	Scene 3	Scene 5
Projected pattern	Number of symbols	2700	2700	2700
Segmentation	Number of square grid points	2578 (95.5%)	2628 (97.3%)	2563 (94.9%)
Conventional method (3×3 color decoding)	Number of 3-D points reconstructed	2477 (91.7%)	2578 (95.5%)	2470 (91.5%)
Proposed method	Number of 3-D points reconstructed	2578 (95.5%)	2632 (97.5%)	2600 (96.3%)

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

and this proves that the conventional method cannot reconstruct all the segmented grid points because of missing neighbors. The last row gives the total number of 3-D points reconstructed using our proposed method and the percentages. These percentages are clearly higher than those of the conventional methods and sometimes even higher than the segmentation result. This is because our proposed method reconstructs symbols that were not extracted in the segmentation stage using the hole-filling algorithm.

Furthermore, the performance of the proposed method was evaluated with a dynamic moving object to prove its applicability in real-time reconstruction. As shown in Fig. 22, the proposed method is capable of accurately reconstructing every frame of a moving object. However, the current frame rate of the proposed method is nearly 3 FPS, and it can be improved by implementing the algorithm using GPU programming.

6.3 *Limitations*

The effect of a surface's own color is the main issue of any color coding and decoding methods. There are many

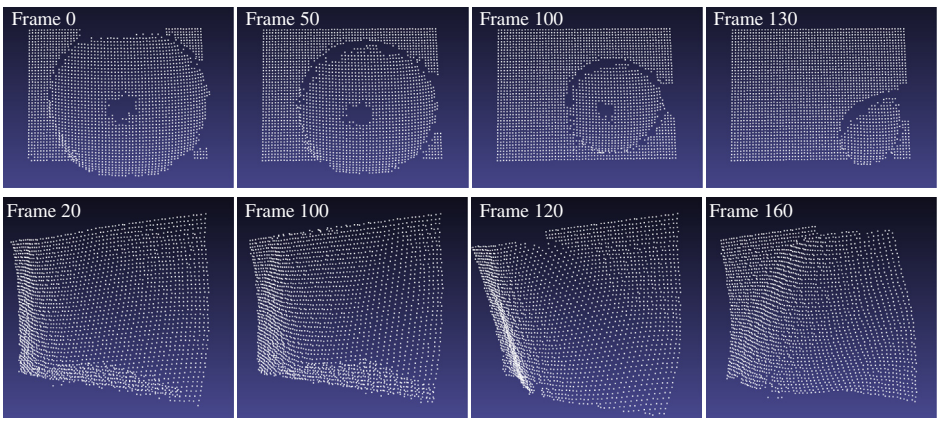


Fig. 22 Reconstruction results of dynamic moving objects. Top row shows a few frames of the reconstructed point clouds on a shrinking balloon, and the bottom row shows a few frames of point clouds on a waving paper sheet. A supplementary video file is available (Video 1: QuickTime, 25.1 MB [URL: http://dx.doi.org/10.1117/1.OE.54.4.043109.1]).

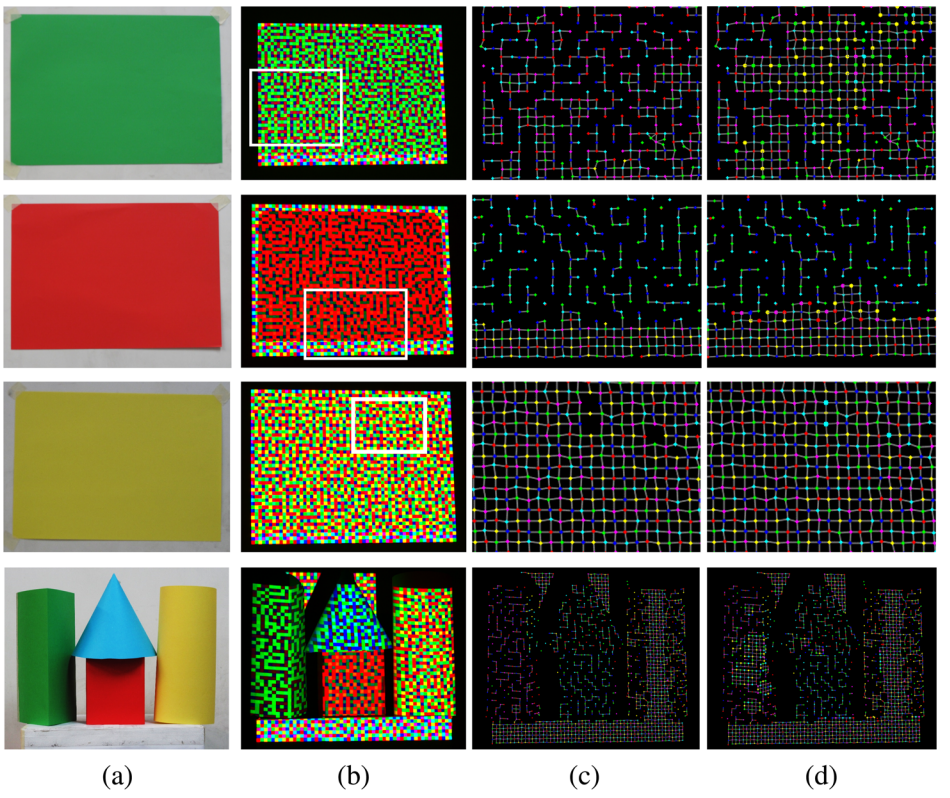


Fig. 23 Checking the performance of the proposed method on objects which have colored surfaces: (a) surfaces with color textures; (b) after projecting the proposed pattern on colored surfaces; (c) decoding results of the conventional method; and (d) decoding results of the proposed method.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms

applications where almost all the materials are of uniform colors and the proposed method can be easily implemented. Even when a color texture exists, choosing the correct pattern colors accordingly or using high contrast color beams enables us to use the color-coded techniques. When these mitigation techniques are not applied, the segmentation process will sometimes fail to extract one or two colors from the captured scenes when color textures exist. If only a few missing symbols exist around the neighborhood, the proposed technique in this paper succeeds in reacquiring the missing grid points and yields better reconstruction results. A comparison between the proposed technique and the conventional method for colored surfaces is depicted in Fig. 23. As shown in the figure, the proposed method works well with the green and yellow colors compared with the blue and red colors.

7 Conclusions

We introduced a new structured light 3-D scanning system based on a pseudorandom array technique. The proposed pattern is a dual pseudorandom array consisting of a binary pattern with the (7×7) window property and a color pattern with the (3×3) window property encoded on top of the binary pattern. Hence, each pattern symbol has two codewords, and we used that property to introduce a dual decoding method. The dual decoding method consists of two steps, (3×3) color decoding and (7×7) binary decoding, and it is capable of decoding symbols even when they have a few missing neighbors. Finally, a hole-filling method that can reacquire the missing symbols by using neighbor information is introduced. We experimentally showed that our proposed method could achieve denser 3-D reconstructions while consuming less time than conventional methods.

As future work, we are planning to improve the efficiency of the system using GPU programming in order to apply the proposed system to reconstruct dynamic scenes in real time.

Acknowledgments

This work was supported by the Industry Core Technology Program granted financial resources from the Ministry of Trade, Industry & Energy, Republic of Korea (No. 10043897).

References

1. J. Salvi et al., "A state of the art in structured light patterns for surface profilometry," *Pattern Recognit.* **43**(8), 2666–2680 (2010).
2. J. Salvi, J. Pagès, and J. Batlle, "Pattern codification strategies in structured light systems," *Pattern Recognit.* **37**(4), 827–849 (2004).
3. F. Forster, "A high-resolution and high accuracy real-time 3D sensor based on structured light," in *3DPVT, Third Int. Symp.*, pp. 208–215, IEEE (2006).
4. P. Fechteler and P. Eisert, "Adaptive colour classification for structured light systems," *IET Comput. Vis.* **3**(2), 49 (2009).
5. M. Maruyama and S. Abe, "Range sensing by projecting multiple slits with random cuts," *IEEE Trans. Pattern Anal. Mach. Intell.* **15**(6), 647–651 (1993).
6. M. Ito and A. Ishii, "A three-level checkerboard pattern (TCP) projection method for curved surface measurement," *Pattern Recognit.* **28**(1), 27–40 (1995).
7. K. L. Boyer and A. C. Kak, "Color-encoded structured light for rapid active ranging," *IEEE Trans. Pattern Anal. Mach. Intell.* **PAMI-9**(1), 14–28 (1987).
8. T. P. Monks, J. N. Carter, and C. H. Shadle, "Colour-encoded structured light for digitisation of real-time 3D data," in *Image Process. its Appl. Int. Conf.*, pp. 327–330, IET (1992).
9. J. Salvi, J. Batlle, and E. Mouaddib, "A robust-coded pattern projection for dynamic 3D scene measurement," *Pattern Recognit. Lett.* **19**(11), 1055–1065 (1998).
10. L. Zhang, B. Curless, and S. M. Seitz, "Rapid shape acquisition using color structured light and multi-pass dynamic programming," in *3DPVT, First Int. Symp.*, pp. 24–36, IEEE (2002).
11. R. Sagawa et al., "Dense 3D reconstruction method using a single pattern for fast moving object," in *2009 IEEE 12th Int. Conf. Comput. Vision* (2009).
12. R. Sagawa et al., "Dense one-shot 3D reconstruction by detecting continuous regions with parallel line projection," in *Proc. IEEE Int. Conf. Comput. Vis.*, pp. 1911–1918 (2011).
13. F. J. MacWilliams and N. J. A. Sloane, "Pseudo-random sequences and arrays," *Proc. IEEE* **64**(12), 1715–1729 (1976).
14. T. Etzion, "Constructions for perfect maps and pseudorandom arrays," *IEEE Trans. Inf. Theory* **34**(5), 1308–1316 (1988).
15. P. M. Griffin, L. S. Narasimhan, and S. R. Yee, "Generation of uniquely encoded light patterns for range data acquisition," *Pattern Recognit.* **25**(6), 609–616 (1992).
16. S. Y. Chen, Y. F. Li, and J. Zhang, "Vision processing for realtime 3-D data acquisition based on coded structured light," *IEEE Trans. Image Process.* **17**(2), 167–176 (2008).
17. R. A. Morano et al., "Structured light using pseudorandom codes," *IEEE Trans. Pattern Anal. Mach. Intell.* **20**(3), 322–327 (1998).
18. J. Pages et al., "An approach to visual servoing based on coded light," in *IEEE Int. Conf. Robot. Autom. ICRA'2006*, pp. 4118–4123 (2006).
19. C. Albitar, P. Graebling, and C. Doignon, "Design of a monochromatic pattern for a robust structured light coding," in *IEEE Int. Conf. ICIP*, **6**, pp. 529–532, IEEE (2007).
20. D. Desjardins and P. Payeur, "Dense stereo range sensing with marching pseudo-random patterns," in *CRV*, pp. 216–226, IEEE (2007).
21. A. Adán et al., "Moving surface extraction based on unordered hexagonal perfect submaps projection: applications to 3D feature tracking," *Image Vis. Comput.* **27**(8), 1083–1096 (2009).
22. U. Wijenayake, S.-I. Choi, and S.-Y. Park, "Combination of color and binary pattern codification for an error correcting M-array technique," in *2012 Ninth Conf. Comput. Robot Vis.*, pp. 139–146, IEEE (2012).
23. U. Wijenayake, S.-H. Baek, and S.-Y. Park, "An error correcting 3D scanning technique using dual pseudorandom arrays," in *2012 Second Int. Conf. 3D Imaging, Model. Process. Vis. Transm.*, pp. 517–523, IEEE (2012).
24. C. Albitar, C. Doignon, and P. Graebling, "Calibration of vision systems based on pseudo-random patterns," in *IEEE/RSJ Int. Conf. IROS*, pp. 321–326, IEEE (2009).
25. A. Marzani et al., "Calibration of a three-dimensional reconstruction system using a structured light source," *Opt. Eng.* **41**(2), 484–492 (2002).
26. H. Anwar, I. Din, and K. Park, "Projector calibration for 3D scanning using virtual target images," *Int. J. Precis. Eng. Manuf.* **13**(1), 125–131 (2012).
27. S. Y. Park and G. G. Park, "Active calibration of camera-projector systems based on planar homography," in *20th Int. Conf. ICPR*, pp. 320–323, IEEE (2010).
28. Z. Zhang, "A flexible new technique for camera calibration," *IEEE Trans. Pattern Anal. Mach. Intell.* **22**(11), 1330–1334 (2000).
29. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, Cambridge University Press (2003).

Udaya Wijenayake received his BSc degree in computer science from the University of Colombo, Sri Lanka, and his MSc degree in electronic engineering and computer science from Kyungpook National University, South Korea, in 2010 and 2013, respectively. He is currently a PhD student in the School of Computer Science and Engineering, Kyungpook National University. His current research interests include 3D reconstruction and 3D motion estimation.

Soon-Yong Park received his BS and MS degrees in electronics engineering from Kyungpook National University, Daegu, Republic of Korea, in 1991 and 1999, and his PhD in electrical and computer engineering from State University of New York at Stony Brook in 2003. From 1993 to 1999, he was a senior research staff at KAERI, Republic of Korea. He is currently an associate professor in the School of Computer Science and Engineering, Kyungpook National University. His research interests include 3D computer and robot vision.

Downloaded From: http://opticalengineering.spiedigitallibrary.org/ on 05/11/2015 Terms of Use: http://spiedl.org/terms